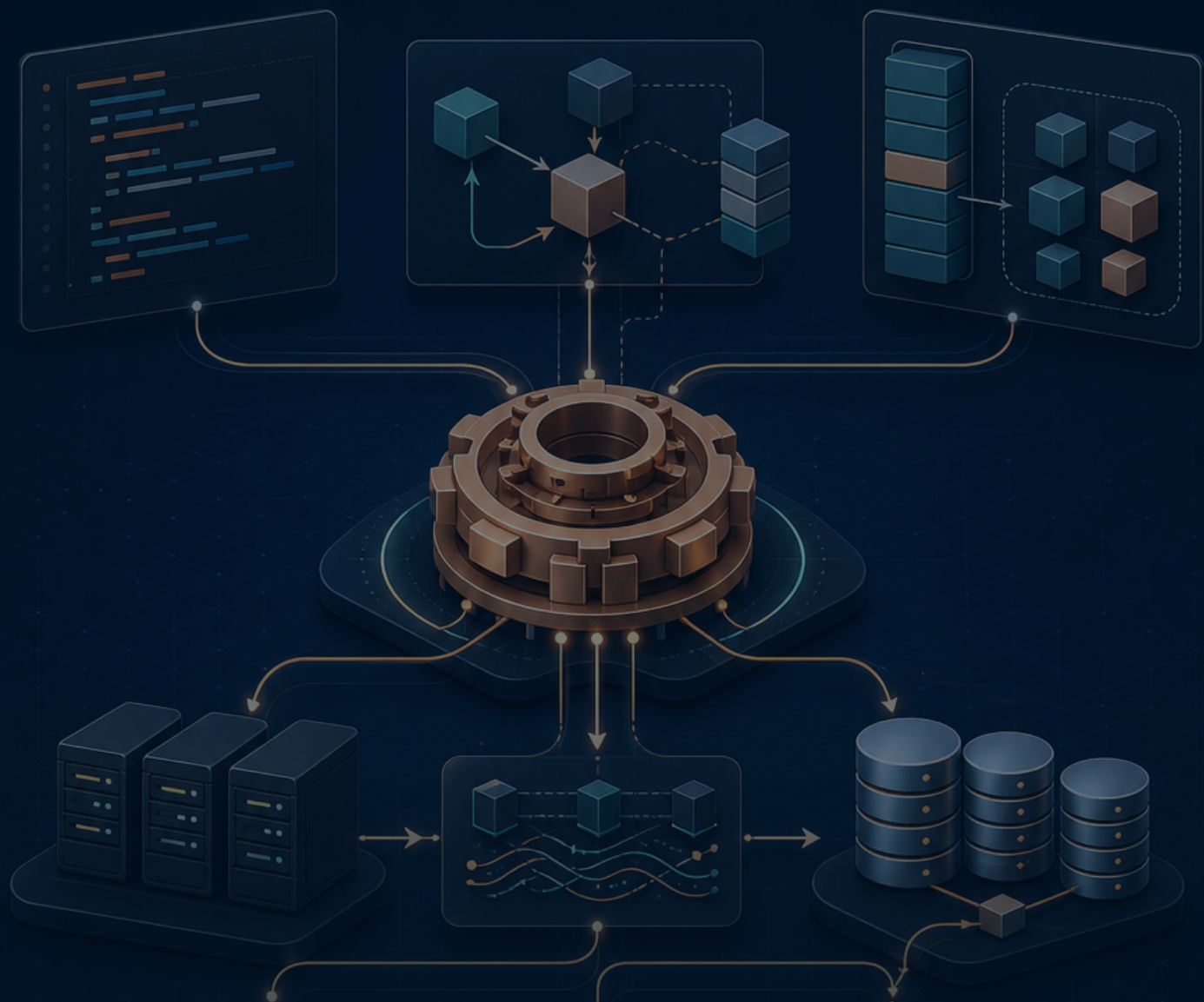




VOLUME 13 / RUST BACKEND MASTERY

完全コード — 演習・解答・用語集・索引

RustNoteの全ファイルを責務ごとに読み、同じ版・command・画面・test・deployで再現する最終巻

10

深掘り単元

20+

実行・解答例

40+

図・表・画面

実務

設計と障害対応

LEARNING MAP

この巻で身に付けること

コードを貼るだけでなく、各fileが何を守り、どの入力で何を表示し、どの失敗を返すかを追跡します。

到達目標

- ✓ 完全構成をゼロから再現できる
- ✓ 全SQLとrouteの責務を説明できる
- ✓ auth・CRUD・asset・deployを一体で検証できる
- ✓ 演習の別解と設計判断をレビューできる

学習地図

01 Cargo.tomlと実行可能な版固定

完全コードの出発点です。Rust 2024、rust-version 1.95、Topcoat/CLI 0.5.0、libsql、Argon2、Serde、Tokio、tracingを固定し、Cargo.lockを含めて同じbuildを再現します。

02 migrations全体とschema不変条件

users、notes、sessions、schema_migrationsを外部キー有効化とtransactionで作ります。認証・owner・title・session期限の不変条件をDDLへ置きます。

03 Config・main・router bootstrap

mainはlogging、Config、DB、migration、AppState、router、startの順だけを担当します。各段階のerrorへcontextを付け、秘密を表示せず、handlerから起動詳細を分離します。

04 domain型とAppError

UserId、NotedId、Email、NoteTitle、NoteState、AppErrorをframeworkから独立したRust型で定義します。未検証文字列をuse caseへ渡さず、HTTP status変換はweb層へ置きます。

05 libSQL NoteRepository完全経路

Repositoryはowner条件付きlist/find/insert/update/toggle/deleteを持ち、SQL値をbindし、rowをNoteへ変換します。0行はNotFound、version不一致はConflictへ分類します。

06 password・session・current_user

Argon2id hash/verifyをspawn_blockingへ隔離し、Topcoat session::start/stop/rotateとsessions tableを同期します。current_userはmemoizeし、期限切れ行を認証なしとして処理します。

07 layout・auth pages・error views

root layoutはdocument、CSS、nav、flash、main slotを描画します。register/loginは項目errorと入力保持、password非保持を実装し、401/404/500は情報を漏らさない一貫した画面へ変換します。

08 CRUD routesと画面の統合

index/create/show/edit/update/toggle/deleteをRoute inventoryどおり実装します。各routeでrequire_user、typed ID、validation、owner Repository、303/400/404/409を同じ順で適用します。

09 CSS・asset・responsive・a11y

app.cssはtoken、layout、form、button、card、status、focus、mobileをまとめ、asset!でcontent hash bundleします。HTML構造と組み合わせてkeyboardとscreen readerを支えます。

10 tests・Docker・fly.toml・CI・runbook

完全コードの最後に、unit/integration/security/smoke、multi-stage Docker、target/assets、Fly port/check、CI gate、deploy/rollback手順を同じ版へ揃えます。

HOW TO USE THIS VOLUME

読むだけで終わらせない進め方

「予想→実行→観察→一か所変更→回帰テスト」を一単位ごとに繰り返します。写経した行数ではなく、入力・処理・出力・失敗を自分の言葉で説明できるかを進捗にします。

1. 予想

実行前に入力型、出力、失敗箇所を予想します。

2. そのまま実行

最初は変更せず、教材と同じ出力が比較します。

3. 一か所変更

値または型を一つだけ変え、差分の理由を説明します。

4. 回帰テスト

直した失敗を自動テストへ残し、再発を防ぎます。



毎回そろえる確認コマンド

● ● ● 単元を終える前の品質確認

```
cargo fmt --check
cargo clippy --all-targets -- -D warnings
cargo test --locked --all-targets
```

対象	この教材の基準	混在を防ぐ理由
Rust	2024 Edition / rust-version 1.95以上	editionとMSRVを明示し、環境差を再現可能にする
Topcoat	crate / CLI ともに 0.5.0固定	early-stageのため、mainブランチの別構文を混ぜない
DB・認証	Turso/libSQL、Argon2id、Topcoat Session	一つのRustプロセスで責務とデータの流れを追える
公開	Docker + Fly.io、HOST=0.0.0.0、PORT=8080	ローカルとコンテナの待受差を明示する

重要な版の前提 Topcoatは公式がearly-stage / experimentalと明記しています。本教材はcrateとCLIを **0.5.0** に固定し、mainブランチの別構文を混ぜません。

UNIT 01

Cargo.tomlと実行可能な版固定

UNIT 01 · CONCEPT

Cargo.tomlと実行可能な版固定

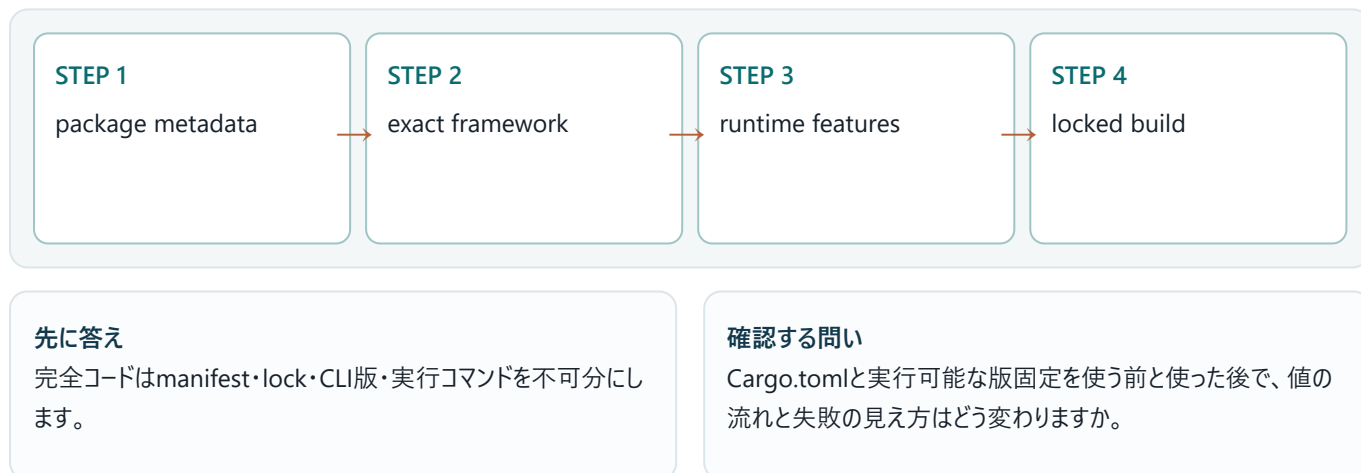
完全コードの出発点です。Rust 2024、rust-version 1.95、Topcoat/CLI 0.5.0、libsql、Argon2、Serde、Tokio、tracingを固定し、Cargo.lockを含めて同じbuildを再現します。

この単元の到達点

- package metadata
- exact framework
- runtime features
- locked build

なぜバックエンドで必要か

教材のコードはこのmanifestとlockで検証します。版を外した場合はdocsとAPI差を確認し、同じ結果を保証しません。



UNIT 01 · MENTAL MODEL

Cargo.tomlと実行可能な版固定を図でつかむ



完全コードの出発点です。Rust 2024、rust-version 1.95、Topcoat/CLI 0.5.0、libsql、Argon2、Serde、Tokio、tracingを固定し、Cargo.lockを含めて同じbuildを再現します。図では左から右へ処理を追います。各箱の入口では入力 の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	package metadata	package metadata — 入力と前提を確認し、境界を曖昧にしない。
2	exact framework	exact framework — 値がどの順で変換されるかを追跡する。
3	runtime features	runtime features — 失敗を再現し、型またはログから原因を特定する。
4	locked build	locked build — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

Cargo.tomlと実行可能な版固定とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 01 · MINIMUM CODE

まず動く最小コード

Cargo.tomlと実行可能な版固定だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
[package]
name="rustnote"
version="0.1.0"
edition="2024"
rust-version="1.95"
[dependencies]
topcoat="=0.5.0"
tokio={version="1",features=["rt-multi-thread","macros","signal"]}
libsql="=0.9.29"
argon2="=0.5.3"
serde={version="1",features=["derive"]}
tracing="0.1"
tracing-subscriber={version="0.3",features=["env-filter"]}
uuid={version="1.24.0",features=["v4"]}
```

●●● 実行コマンド

```
cargo check --locked --all-targets
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順で進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 01・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
[package]	この行が受け取る値、作る値、副作用の有無を確認します。
name="rustnote"	この行が受け取る値、作る値、副作用の有無を確認します。
version="0.1.0"	この行が受け取る値、作る値、副作用の有無を確認します。
edition="2024"	この行が受け取る値、作る値、副作用の有無を確認します。
rust-version="1.95"	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力がpackage metadataの条件を満たすか確認する。
- ✓ exact frameworkを実行し、途中の型と状態を追う。
- ✓ runtime featuresで失敗を成功経路から分離する。
- ✓ locked buildにより、出力と副作用を検証する。

型を声に出す Cargo.tomlと実行可能な版固定に関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 01・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
cargo check --locked --all-targets → success
```

見える結果
端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと
型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 01・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
topcoat:web/session/assets; tokio:async runtime; libsql:Turso; argon2:password; serde:form DTO;
tracing:logs; uuid:request/idempotency IDs
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

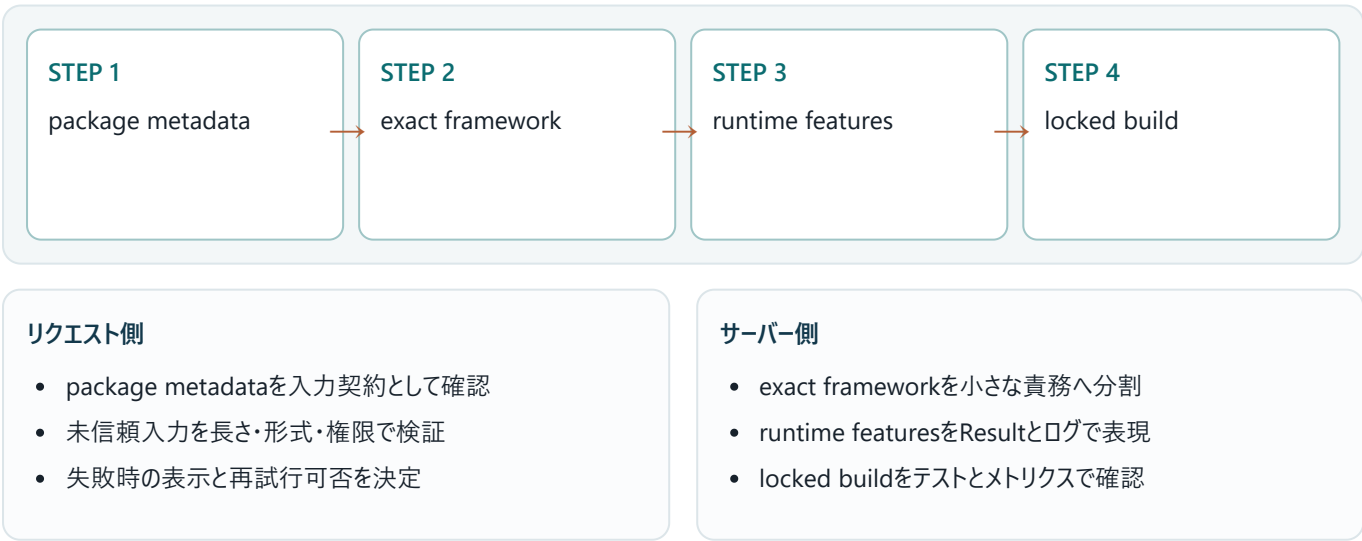
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 01・BACKEND APPLICATION

バックエンドのどこで使うか

教材のコードはこのmanifestとlockで検証します。版を外した場合はdocsとAPI差を確認し、同じ結果を保証しません。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。

- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 01・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	package metadataに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	exact frameworkの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	runtime featuresまたはlocked buildの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 01・DEBUGGING

Cargo.tomlと実行可能な版固定を再現して切り分ける

Cargo.tomlと実行可能な版固定の問題を、環境・入力・型・副作用の順に切り分けます。

1. STEP 1 — エラー全文と再現コマンドを保存する。対象: package metadata
2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: exact framework
3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: runtime features
4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: locked build

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	package metadata	package metadata — 入力と前提を確認し、境界を曖昧にしない。
2	exact framework	exact framework — 値がどの順で変換されるかを追跡する。
3	runtime features	runtime features — 失敗を再現し、型またはログから原因を特定する。
4	locked build	locked build — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 完全コードはmanifest・lock・CLI版・実行コマンドを不可分にします。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 01 · PRACTICE

手を動かす演習

依存を一つずつ削除して、どのmoduleがcompileできなくなるか責務表を作ってください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 01・SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
topcoat:web/session/assets; tokio:async runtime; libsql:Turso; argon2:password; serde:form DTO;
tracing:logs; uuid:request/idempotency IDs
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 完全コードはmanifest・lock・CLI版・実行コマンドを不可分にします。
- ✓ Cargo.tomlと実行可能な版固定とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 02

migrations全体とschema不変条件

UNIT 02 · CONCEPT

migrations全体とschema不変条件

users、notes、sessions、schema_migrationsを外部キー有効化とtransactionで作ります。認証・owner・title・session期限の不変条件をDDLへ置きます。

この単元の到達点

- users
- notes
- sessions
- indexes/migrations

なぜバックエンドで必要か

起動時runnerは適用済みversionを読み、未適用SQLだけをtransactionで実行します。本番はrelease commandから一度だけ実行します。



先に答え

アプリ検証をすり抜けてもDB制約が不正状態を拒否します。

確認する問い

migrations全体とschema不変条件を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

UNIT 02 · MENTAL MODEL

migrations全体とschema不変条件を図でつかむ



users、notes、sessions、schema_migrationsを外部キー有効化とtransactionで作ります。認証・owner・title・session期限の不変条件をDDLへ置きます。図では左から右へ処理を追います。各箱の入口では入力のと信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	users	users — 入力と前提を確認し、境界を曖昧にしない。
2	notes	notes — 値がどの順で変換されるかを追跡する。
3	sessions	sessions — 失敗を再現し、型またはログから原因を特定する。
4	indexes/migrations	indexes/migrations — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

migrations全体とschema不変条件とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 02・MINIMUM CODE まず動く最小コード

migrations全体とschema不変条件だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
PRAGMA foreign_keys=ON;
CREATE TABLE users(id INTEGER PRIMARY KEY,email TEXT NOT NULL UNIQUE,password_hash TEXT NOT NULL,created_at
TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP);
CREATE TABLE notes(id INTEGER PRIMARY KEY,user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE
CASCADE,title TEXT NOT NULL CHECK(length(trim(title)) BETWEEN 1 AND 80),body TEXT NOT NULL DEFAULT '',done
INTEGER NOT NULL DEFAULT 0 CHECK(done IN(0,1)),version INTEGER NOT NULL DEFAULT 1,created_at TEXT NOT NULL
DEFAULT CURRENT_TIMESTAMP,updated_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP);
CREATE TABLE sessions(token_hash BLOB PRIMARY KEY,user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE
CASCADE,expires_at TEXT NOT NULL,created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP);
CREATE INDEX idx_notes_user_created ON notes(user_id,created_at DESC,id DESC);
```

●●● 実行コマンド

```
turso db shell rustnote-dev < migrations/0001_init.sql
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 02・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
PRAGMA foreign_keys=ON;	この行が受け取る値、作る値、副作用の有無を確認します。
CREATE TABLE users(id INTEGER PRIMARY KEY,email TEXT NOT NULL UNIQUE,passw	SQLの責務を一文で読み、プレースホルダと制約で入力を守ります。
CREATE TABLE notes(id INTEGER PRIMARY KEY,user_id INTEGER NOT NULL REFEREN	SQLの責務を一文で読み、プレースホルダと制約で入力を守ります。
CREATE TABLE sessions(token_hash BLOB PRIMARY KEY,user_id INTEGER NOT NULL	SQLの責務を一文で読み、プレースホルダと制約で入力を守ります。
CREATE INDEX idx_notes_user_created ON notes(user_id,created_at DESC,id DE	SQLの責務を一文で読み、プレースホルダと制約で入力を守ります。

実行時に起きる順序

- ✓ 入力がusersの条件を満たすか確認する。
- ✓ notesを実行し、途中の型と状態を追う。
- ✓ sessionsで失敗を成功経路から分離する。
- ✓ indexes/migrationsにより、出力と副作用を検証する。

型を声に出す migrations全体とschema不変条件に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 02・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

●●● 実行結果

migration version 1 applied
foreign keys enabled
indexes created

見える結果
端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと
型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 02・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
INSERT INTO notes(user_id,title) VALUES(999,'missing owner'); -- FK fail
INSERT INTO notes(user_id,title) VALUES(1,''); -- CHECK fail
INSERT INTO users(email,password_hash) VALUES('same@example.com','x'); -- second time UNIQUE fail
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 02・BACKEND APPLICATION

バックエンドのどこで使うか

起動時runnerは適用済みversionを読み、未適用SQLだけをtransactionで実行します。本番はrelease commandから一度だけ実行します。



リクエスト側

- usersを入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- notesを小さな責務へ分割
- sessionsをResultとログで表現
- indexes/migrationsをテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 02 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	usersに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	notesの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	sessionsまたはindexes/migrationsの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 02・DEBUGGING

migrations全体とschema不変条件を再現して切り分ける

migrations全体とschema不変条件の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: users
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: notes
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: sessions
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: indexes/migrations

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	users	users — 入力と前提を確認し、境界を曖昧にしない。
2	notes	notes — 値がどの順で変換されるかを追跡する。
3	sessions	sessions — 失敗を再現し、型またはログから原因を特定する。
4	indexes/migrations	indexes/migrations — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 アプリ検証をすり抜けてもDB制約が不正状態を拒否します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 02 ・ PRACTICE

手を動かす演習

制約違反を起こすINSERTを各一つ作り、DBが拒否することを確認してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 02 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
INSERT INTO notes(user_id,title) VALUES(999,'missing owner'); -- FK fail
INSERT INTO notes(user_id,title) VALUES(1,''); -- CHECK fail
INSERT INTO users(email,password_hash) VALUES('same@example.com','x'); -- second time UNIQUE fail
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ アプリ検証をすり抜けてもDB制約が不正状態を拒否します。
- ✓ migrations全体とschema不変条件とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 03

Config・main・router bootstrap

UNIT 03 · CONCEPT

Config・main・router bootstrap

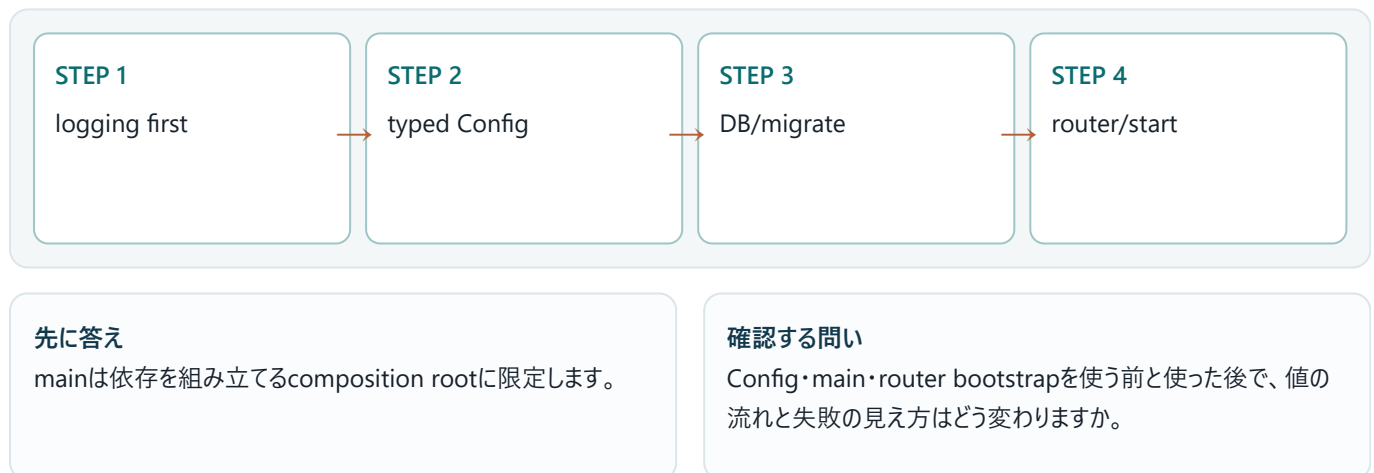
mainはlogging、Config、DB、migration、AppState、router、startの順だけを担当します。各段階のerrorへcontextを付け、秘密を表示せず、handlerから起動詳細を分離します。

この単元の到達点

- logging first
- typed Config
- DB/migrate
- router/start

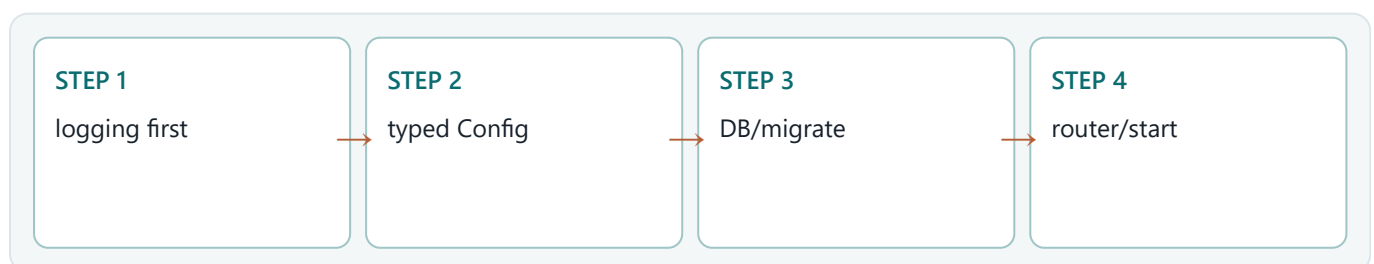
なぜバックエンドで必要か

Topcoat startはHOST/PORTを読みます。Flyではenvに0.0.0.0/8080を設定し、AppStateにはSend+SyncなDB handleと検証済みConfigだけを置きます。



UNIT 03 · MENTAL MODEL

Config・main・router bootstrapを図でつかむ



mainはlogging、Config、DB、migration、AppState、router、startの順だけを担当します。各段階のerrorへcontextを付け、秘密を表示せず、handlerから起動詳細を分離します。図では左から右へ処理を追います。各箱の入口では入力型と信頼度、出口では結果の型

と副作用を確認します。

順序	観察対象	初心者が確認すること
1	logging first	logging first — 入力と前提を確認し、境界を曖昧にしない。
2	typed Config	typed Config — 値がどの順で変換されるかを追跡する。
3	DB/migrate	DB/migrate — 失敗を再現し、型またはログから原因を特定する。
4	router/start	router/start — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

Config・main・router bootstrapとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 03・MINIMUM CODE

まず動く最小コード

Config・main・router bootstrapだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[tokio::main]
async fn main()->Result<(),Box<dyn std::error::Error>>{
    tracing_subscriber::fmt().with_env_filter(EnvFilter::from_default_env()).init();
    let config=Config::load()?;
    let db=open_database(&config).await?;
    migrate(&db).await?;
    let state=AppState{config,db};
    let
    router=Router::builder().discover().cookies().sessions(SessionConfig::default()).assets(AssetBundle::load()?)
    ).app_context(state).build();
    topcoat::start(router).await?;
    Ok(())
}
```

● ● ● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 03 ・ LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[tokio::main]</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>async fn main()-> Result<(),Box<dyn std::error::Error>>{</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>tracing_subscriber::fmt().with_env_filter(EnvFilter::from_default_env()).i</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>let config=Config::load()?;</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>let db=open_database(&config).await?;</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。

実行時に起きる順序

- ✓ 入力がlogging firstの条件を満たすか確認する。
- ✓ typed Configを実行し、途中の型と状態を追う。
- ✓ DB/migrateで失敗を成功経路から分離する。
- ✓ router/startにより、出力と副作用を検証する。

型を声に出す Config・main・router bootstrapに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 03 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
INFO migration current=1
INFO listening host=0.0.0.0 port=8080
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 03 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
struct Secret(String);impl std::fmt::Debug for Secret{fn fmt(&self,f:&mut std::fmt::Formatter<'_>)->std::fmt::Result{f.write_str("[REDACTED]")}}
struct Config{db_url:String,db_token:Secret}
impl Config{fn load()->Result<Self,ConfigError>
{Ok(Self{db_url:required("TURSO_DATABASE_URL"?),db_token:Secret(required("TURSO_AUTH_TOKEN"?))}}}}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

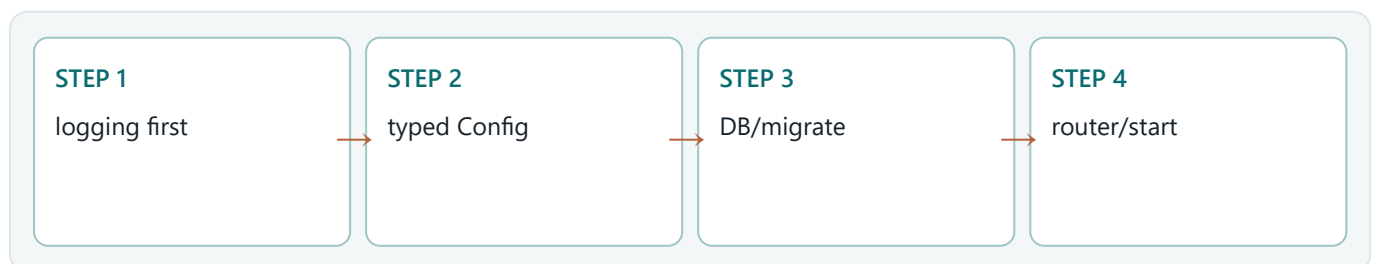
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 03 · BACKEND APPLICATION

バックエンドのどこで使うか

Topcoat startはHOST/PORTを読みます。Flyではenvに0.0.0.0/8080を設定し、AppStateにはSend+SyncなDB handleと検証済みConfigだけを置きます。



リクエスト側

- logging firstを入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- typed Configを小さな責務へ分割
- DB/migrateをResultとログで表現
- router/startをテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 03 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	logging firstに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	typed Configの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	DB/migrateまたはrouter/startの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。

✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 03・DEBUGGING

Config・main・router bootstrapを再現して切り分ける

Config・main・router bootstrapの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: logging first
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: typed Config
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: DB/migrate
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: router/start

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	logging first	logging first — 入力と前提を確認し、境界を曖昧にしない。
2	typed Config	typed Config — 値がどの順で変換されるかを追跡する。
3	DB/migrate	DB/migrate — 失敗を再現し、型またはログから原因を特定する。
4	router/start	router/start — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 mainは依存を組み立てるcomposition rootに限定します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 03 ・ PRACTICE

手を動かす演習

Config::loadでURL/token/host/portを検証し、tokenのDebugをredactしてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 03 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
struct Secret(String);impl std::fmt::Debug for Secret{fn fmt(&self,f:&mut std::fmt::Formatter<'_>)->std::fmt::Result{f.write_str("[REDACTED]")}}
struct Config{db_url:String,db_token:Secret}
impl Config{fn load()->Result<Self,ConfigError>
{Ok(Self{db_url:required("TURSO_DATABASE_URL"?),db_token:Secret(required("TURSO_AUTH_TOKEN"?))})}}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ mainは依存を組み立てるcomposition rootに限定します。
- ✓ Config・main・router bootstrapとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 04

domain型とAppError

UNIT 04 · CONCEPT

domain型とAppError

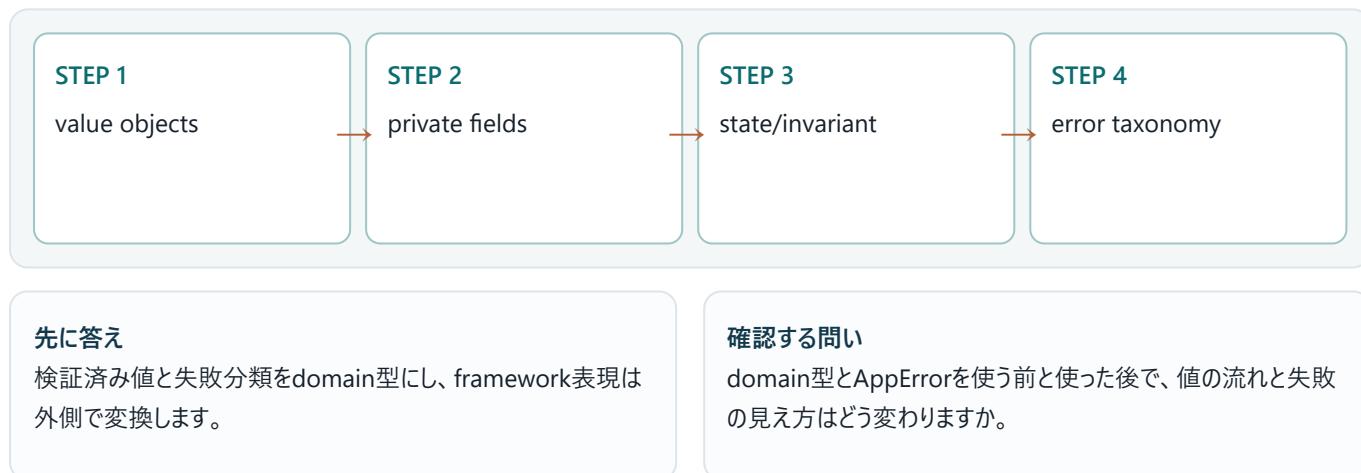
UserId、NotelId、Email、NoteTitle、NoteState、AppErrorをframeworkから独立したRust型で定義します。未検証文字列をuse caseへ渡さず、HTTP status変換はweb層へ置きます。

この単元の到達点

- value objects
- private fields
- state/invariant
- error taxonomy

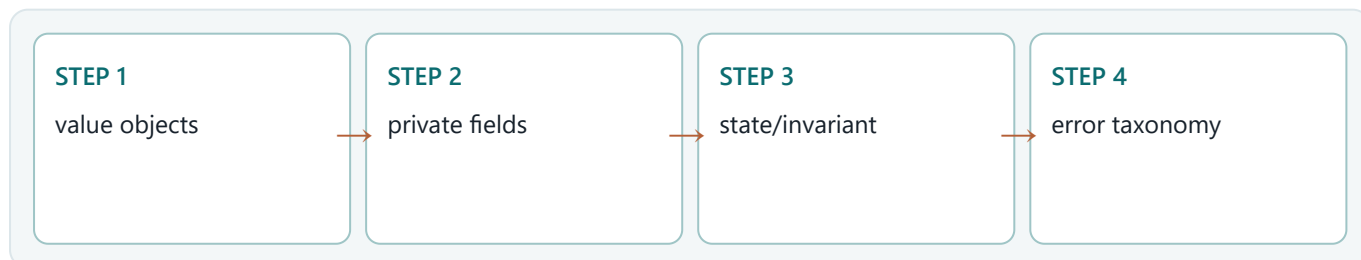
なぜバックエンドで必要か

errorのDisplayは内部診断、利用者向けmessageはweb mappingで定型化します。source連鎖とrequest_idはlogへ残します。



UNIT 04 · MENTAL MODEL

domain型とAppErrorを図でつかむ



UserId、NotelId、Email、NoteTitle、NoteState、AppErrorをframeworkから独立したRust型で定義します。未検証文字列をuse caseへ渡さず、HTTP status変換はweb層へ置きます。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	value objects	value objects — 入力と前提を確認し、境界を曖昧にしない。
2	private fields	private fields — 値がどの順で変換されるかを追跡する。
3	state/invariant	state/invariant — 失敗を再現し、型またはログから原因を特定する。
4	error taxonomy	error taxonomy — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

domain型とAppErrorとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 04 · MINIMUM CODE
まず動く最小コード

domain型とAppErrorだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[derive(Clone, Copy, Debug, Eq, PartialEq)]struct UserId(i64);
#[derive(Clone, Copy, Debug, Eq, PartialEq)]struct NoteId(i64);
struct NoteTitle(String); impl NoteTitle {fn parse(raw: &str) -> Result<Self, ValidationError> {let
s=raw.trim(); let n=s.chars().count(); if n==0 {return Err(ValidationError::Empty)} if n>80 {return
Err(ValidationError::TooLong {max:80, actual:n})} Ok(Self(s.into()))} }
#[derive(Debug)] enum AppError {Validation(ValidationError), Unauthorized, NotFound, Conflict, Unavailable}
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 04 · LINE BY LINE
コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[derive(Clone, Copy, Debug, Eq, PartialEq)] struct UserId(i64);</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>#[derive(Clone, Copy, Debug, Eq, PartialEq)] struct NoteId(i64);</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>struct NoteTitle(String); impl NoteTitle { fn parse(raw: &str) -> Result<Self, Va</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>#[derive(Debug)] enum AppError { Validation(ValidationError), Unauthorized, Not</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力値がvalue objectsの条件を満たすか確認する。
- ✓ private fieldsを実行し、途中の型と状態を追う。
- ✓ state/invariantで失敗を成功経路から分離する。
- ✓ error taxonomyにより、出力と副作用を検証する。

型を声に出す domain型とAppErrorに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 04 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
NoteTitle::parse(" Rust ") → Ok("Rust")
empty → Validation::Empty
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 04 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
fn status(e:&AppError)->u16{match
e{AppError::Validation(_)=>400,AppError::Unauthorized=>401,AppError::NotFound=>404,AppError::Conflict=>409,AppError::Unavailable=>503}}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

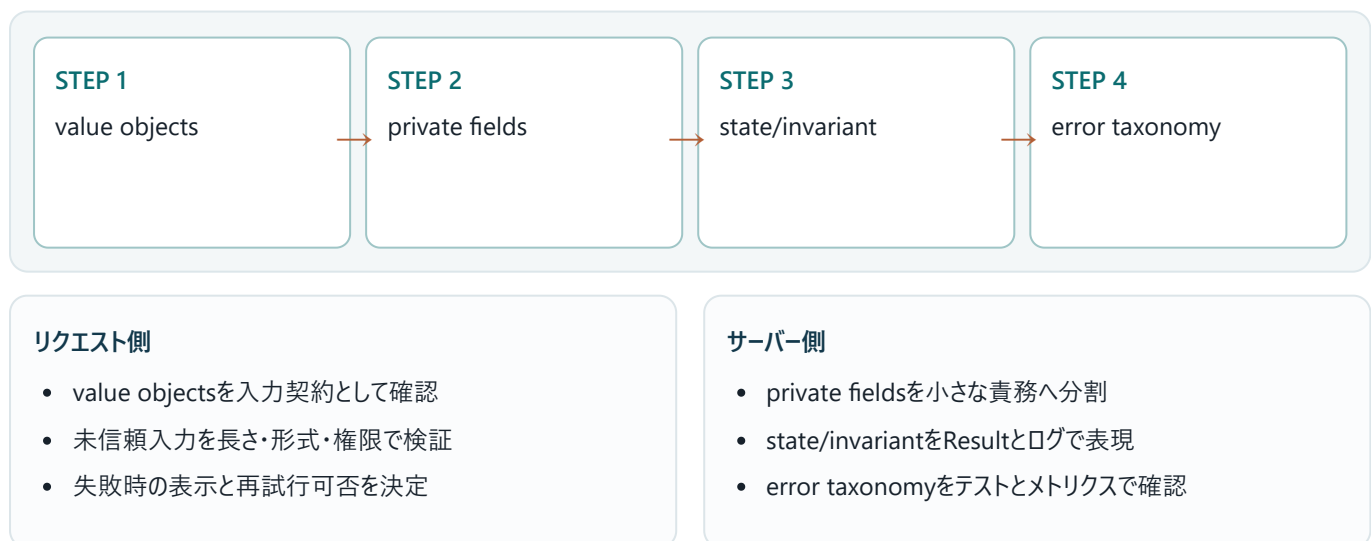
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 04 · BACKEND APPLICATION

バックエンドのどこで使うか

errorのDisplayは内部診断、利用者向けmessageはweb mappingで定型化します。source連鎖とrequest_idはlogへ残します。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。

- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 04 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	value objectsに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	private fieldsの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	state/invariantまたはerror taxonomyの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 04 · DEBUGGING

domain型とAppErrorを再現して切り分ける

domain型とAppErrorの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: value objects
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: private fields
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: state/invariant
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: error taxonomy

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	value objects	value objects — 入力と前提を確認し、境界を曖昧にしない。
2	private fields	private fields — 値がどの順で変換されるかを追跡する。
3	state/invariant	state/invariant — 失敗を再現し、型またはログから原因を特定する。
4	error taxonomy	error taxonomy — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 検証済み値と失敗分類をdomain型にし、framework表現は外側で変換します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 04 · PRACTICE

手を動かす演習

AppError各variantを400/302または401/404/409/503へ変換する表と関数を作ってください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 04 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
fn status(e:&AppError)->u16{match
e{AppError::Validation(_)=>400,AppError::Unauthorized=>401,AppError::NotFound=>404,AppError::Conflict=>409,AppError::Unavailable=>503}}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 検証済み値と失敗分類をdomain型にし、framework表現は外側で変換します。
- ✓ domain型とAppErrorとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 05

libSQL NoteRepository完全経路

UNIT 05 · CONCEPT

libSQL NoteRepository完全経路

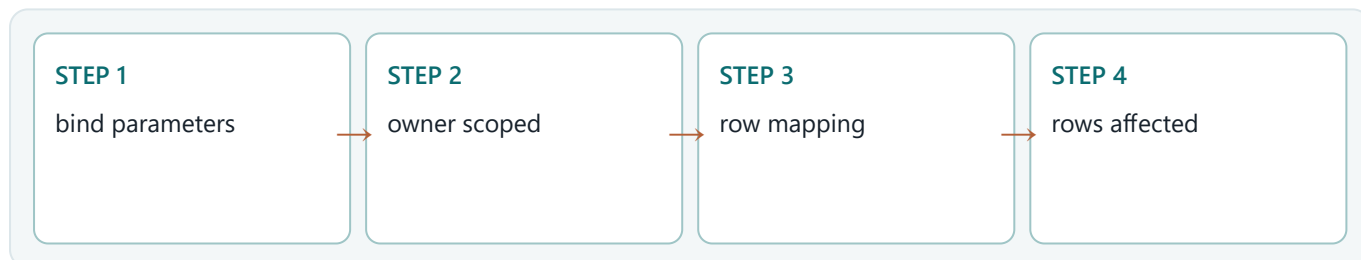
Repositoryはowner条件付きlist/find/insert/update/toggle/deleteを持ち、SQL値をbindし、rowをNoteへ変換します。0行はNotFound、version不一致はConflictへ分類します。

この単元の到達点

- bind parameters
- owner scoped
- row mapping
- rows affected

なぜバックエンドで必要か

Connection取得、transaction、query timeout、operation spanをRepository内部で統一します。libsql error文字列をHTMLへ出しません。



先に答え

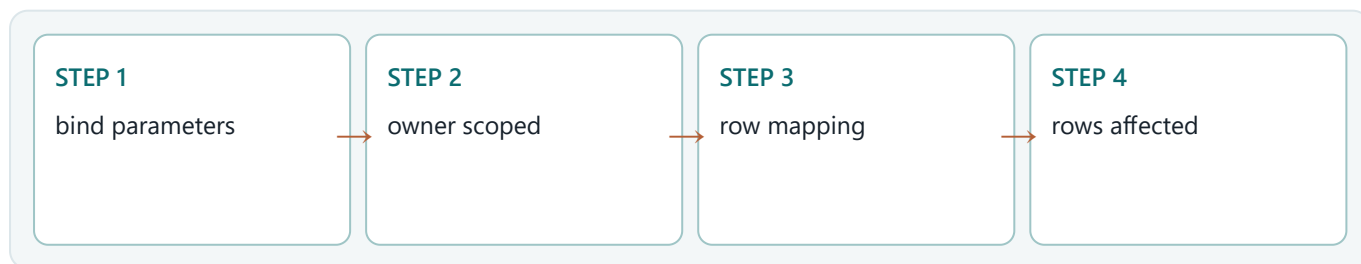
Repositoryの全SQLへowner bindを含め、外へdomain型と分類済みerrorだけを返します。

確認する問い

libSQL NoteRepository完全経路を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

UNIT 05 · MENTAL MODEL

libSQL NoteRepository完全経路を図でつかむ



Repositoryはowner条件付きlist/find/insert/update/toggle/deleteを持ち、SQL値をbindし、rowをNoteへ変換します。0行はNotFound、version不一致はConflictへ分類します。図では左から右へ処理を追います。各箱の入口では入力の種類と信頼度、出口では結

果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	bind parameters	bind parameters — 入力と前提を確認し、境界を曖昧にしない。
2	owner scoped	owner scoped — 値がどの順で変換されるかを追跡する。
3	row mapping	row mapping — 失敗を再現し、型またはログから原因を特定する。
4	rows affected	rows affected — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

libSQL NoteRepository完全経路とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 05 · MINIMUM CODE

まず動く最小コード

libSQL NoteRepository完全経路だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
async fn find_owned(&self, user: UserId, id: NoteId) -> Result<Option<Note>, RepoError> {
    let mut rows = self.conn.query("SELECT id, title, body, done, version FROM notes WHERE id=?1 AND user_id=?2", params!
    [id.0, user.0]).await?;
    rows.next().await?.map(map_note).transpose()
}
async fn delete_owned(&self, user: UserId, id: NoteId) -> Result<(), AppError> {
    let n = self.conn.execute("DELETE FROM notes WHERE id=?1 AND user_id=?2", params!
    [id.0, user.0]).await?;
    if n == 0 { Err(AppError::NotFound) } else { Ok(()) }
}
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 05 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<pre>async fn find_owned(&self,user:UserId,id:NotId)- >Result<Option<Note>,Repo</pre>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<pre>async fn delete_owned(&self,user:UserId,id:NotId)- >Result<(),AppError>{le</pre>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
libSQL NoteRepository完全経路	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力が入力契約の条件を満たすか確認する。
- ✓ owner scopedを実行し、途中の型と状態を追う。
- ✓ row mappingで失敗を成功経路から分離する。
- ✓ rows affectedにより、出力と副作用を検証する。

型を声に出す libSQL NoteRepository完全経路に関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 05 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
owner find → Some(Note)
other owner → None
delete own → affected=1
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 05 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
let changed=execute(UPDATE_WITH_VERSION,...).await?.if changed==1{return Ok(())}if
find_owned(user,id).await?.is_some(){Err(AppError::Conflict)}else{Err(AppError::NotFound)}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

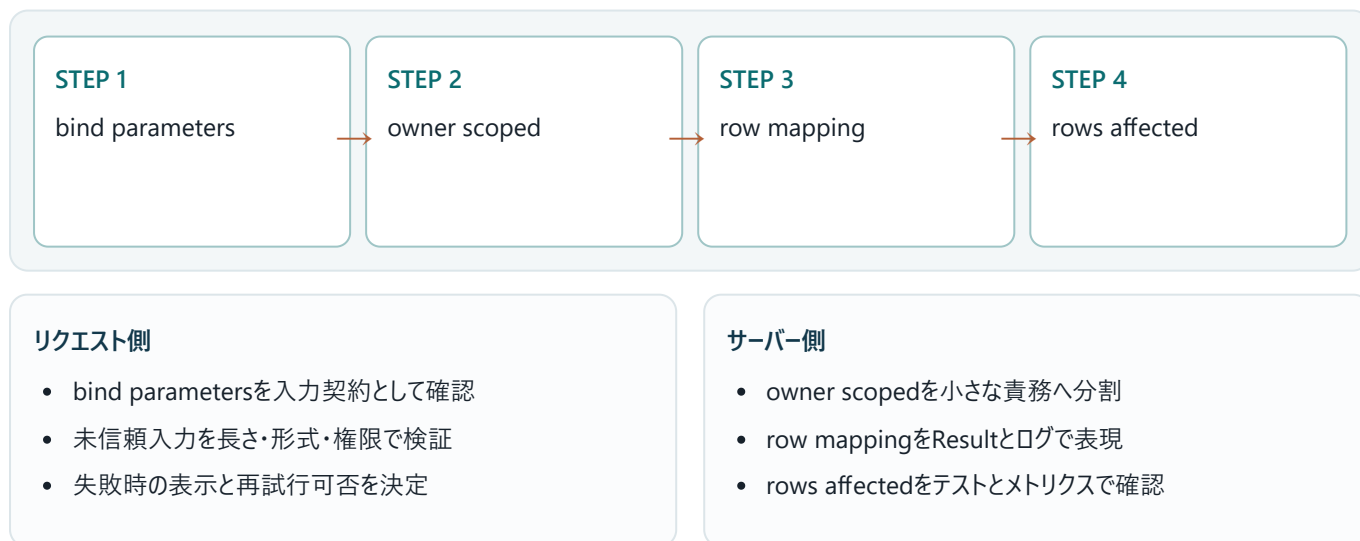
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 05 · BACKEND APPLICATION

バックエンドのどこで使うか

Connection取得、transaction、query timeout、operation spanをRepository内部で統一します。libsql error文字列をHTMLへ出しません。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。

- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 05 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	bind parametersに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	owner scopedの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	row mappingまたはrows affectedの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 05・DEBUGGING

libSQL NoteRepository完全経路を再現して切り分ける

libSQL NoteRepository完全経路の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: bind parameters
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: owner scoped
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: row mapping
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: rows affected

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	bind parameters	bind parameters — 入力と前提を確認し、境界を曖昧にしない。
2	owner scoped	owner scoped — 値がどの順で変換されるかを追跡する。
3	row mapping	row mapping — 失敗を再現し、型またはログから原因を特定する。
4	rows affected	rows affected — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 Repositoryの全SQLへowner bindを含め、外へdomain型と分類済みerrorだけを返します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 05 ・ PRACTICE

手を動かす演習

updateのversion条件で0行の場合、owner存在確認によりConflictとNotFoundを区別してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 05 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
let changed=execute(UPDATE_WITH_VERSION,...).await?;if changed==1{return Ok({})}if
find_owned(user,id).await?.is_some(){Err(AppError::Conflict)}else{Err(AppError::NotFound)}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ Repositoryの全SQLへowner bindを含め、外へdomain型と分類済みerrorだけを返します。
- ✓ libSQL NoteRepository完全経路とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 06

password・session・current_user

UNIT 06 ・ CONCEPT

password・session・current_user

Argon2id hash/verifyをspawn_blockingへ隔離し、Topcoat session::start/stop/rotateとsessions tableを同期します。current_userはmemoizeし、期限切れ行を認証なしとして処理します。

この単元の到達点

- Argon2 blocking
- session hash
- expiry/revoke
- memoized current_user

なぜバックエンドで必要か

password hash、session raw token、Turso tokenをlogへ出しません。login rate limitとsecurity testsをrelease gateにします。



UNIT 06 ・ MENTAL MODEL

password・session・current_userを図でつかむ



Argon2id hash/verifyをspawn_blockingへ隔離し、Topcoat session::start/stop/rotateとsessions tableを同期します。current_userはmemoizeし、期限切れ行を認証なしとして処理します。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	Argon2 blocking	Argon2 blocking — 入力と前提を確認し、境界を曖昧にしない。
2	session hash	session hash — 値がどの順で変換されるかを追跡する。
3	expiry/revoke	expiry/revoke — 失敗を再現し、型またはログから原因を特定する。
4	memoized current_user	memoized current_user — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

password・session・current_userとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 06 ・ MINIMUM CODE

まず動く最小コード

password・session・current_userだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[memoize]async fn current_user(cx:&Cx)->Result<Option<User>>{let Some(hash)=session::token_hash(cx).await?
else{return Ok(None)};repo(cx).find_unexpired_user(&hash).await.map_err(Into::into)}
#[route(POST "/login")]async fn login(cx:&Cx,Form(input):Form<LoginInput>)->Result<SeeOther>{let
user=verify_credentials(&input).await?;let
s=session::start(cx).await?;repo(cx).insert_session(user.id,&s).await?;Ok(see_other("/notes"))}
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 06 ・ LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[memoize]async fn current_user(cx:&Cx)->Result<Option<User>>{let Some(has</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>#[route(POST "/login")]async fn login(cx:&Cx,Form(input):Form<LoginInput>)</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>password・session・current_user</code>	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力がArgon2 blockingの条件を満たすか確認する。
- ✓ session hashを実行し、途中の型と状態を追う。
- ✓ expiry/revokeで失敗を成功経路から分離する。
- ✓ memoized current_userにより、出力と副作用を検証する。

型を声に出す password・session・current_userに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値の一つを変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 06 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
login success → hardened cookie + DB hash + 303
wrong → generic 400/401
logout → cookie + DB row removed
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 06 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
let s1=login(user).await;let s2=login(user).await;logout_current(&s1).await;assert!(
(!account(&s1).await.is_ok());assert!(account(&s2).await.is_ok());revoke_all(user.id).await;assert!(
(!account(&s2).await.is_ok());
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

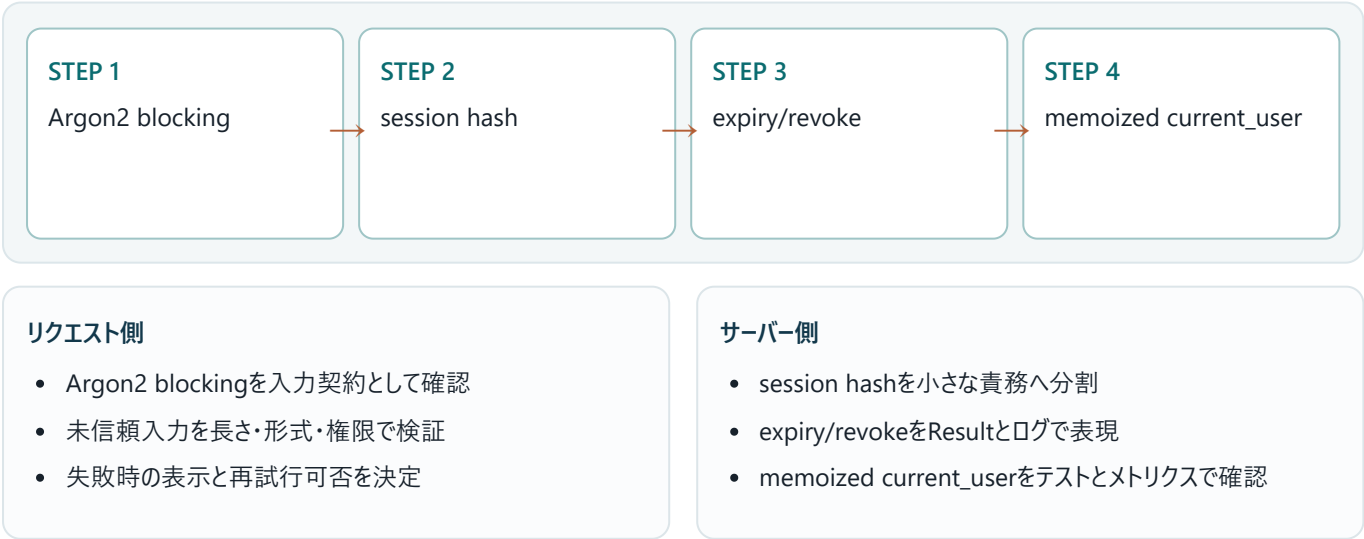
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 06 · BACKEND APPLICATION

バックエンドのどこで使うか

password hash、session raw token、Turso tokenをlogへ出しません。login rate limitとsecurity testsをrelease gateにします。



- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 06 ・ FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	Argon2 blockingに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	session hashの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	expiry/revokeまたはmemoized current_userの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 06・DEBUGGING

password・session・current_userを再現して切り分ける

password・session・current_userの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: Argon2 blocking
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: session hash
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: expiry/revoke
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: memoized current_user

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	Argon2 blocking	Argon2 blocking — 入力と前提を確認し、境界を曖昧にしない。
2	session hash	session hash — 値がどの順で変換されるかを追跡する。
3	expiry/revoke	expiry/revoke — 失敗を再現し、型またはログから原因を特定する。
4	memoized current_user	memoized current_user — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 資格情報検証、cookie token、DB hash、期限、revokeを一つの認証経路にします。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 06 ・ PRACTICE

手を動かす演習

logout後のcookie再利用と、他端末全logoutをintegration testしてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 06 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
let s1=login(user).await;let s2=login(user).await;logout_current(&s1).await;assert!(
!account(&s1).await.is_ok());assert!(account(&s2).await.is_ok());revoke_all(user.id).await;assert!(
!account(&s2).await.is_ok());
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 資格情報検証、cookie token、DB hash、期限、revokeを一つの認証経路にします。
- ✓ password・session・current_userとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 07

layout・auth pages・error views

UNIT 07・CONCEPT

layout・auth pages・error views

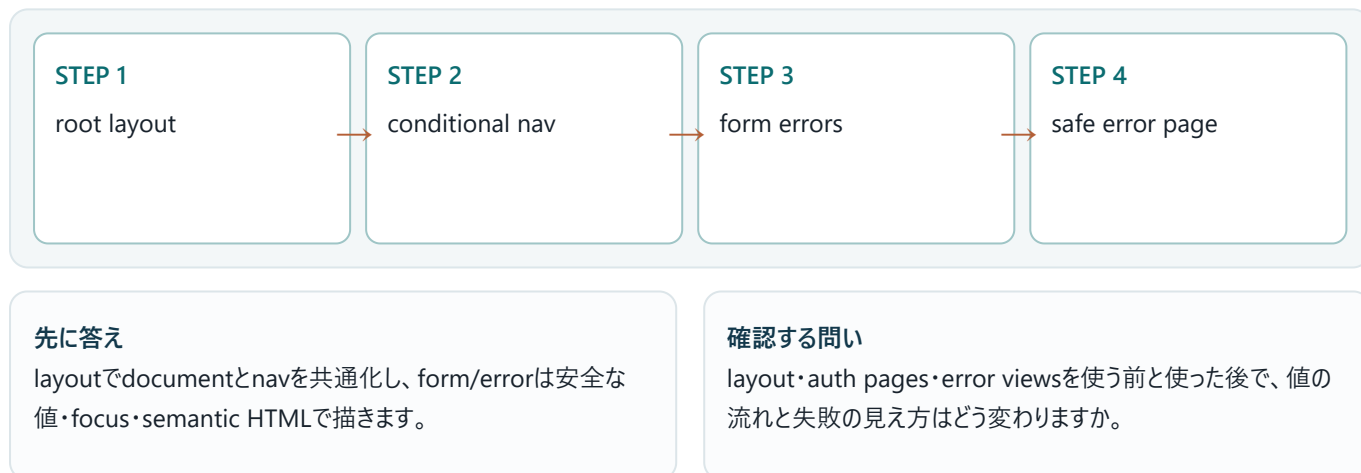
root layoutはdocument、CSS、nav、flash、main slotを描画します。register/loginは項目errorと入力保持、password非保持を実装し、401/404/500は情報を漏らさない一貫した画面へ変換します。

この単元の到達点

- root layout
- conditional nav
- form errors
- safe error page

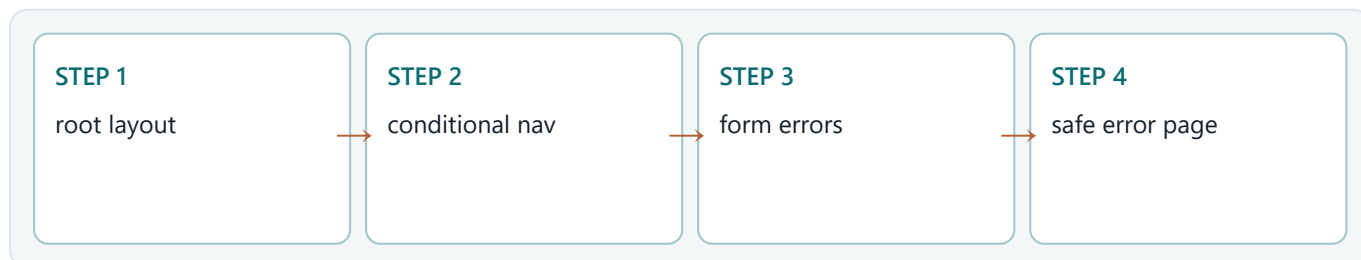
なぜバックエンドで必要か

component引数はescapeされ、raw HTMLを作りません。error detailはlogへ、画面は安全な定型文と再試行導線を出します。



UNIT 07・MENTAL MODEL

layout・auth pages・error viewsを図でつかむ



root layoutはdocument、CSS、nav、flash、main slotを描画します。register/loginは項目errorと入力保持、password非保持を実装し、401/404/500は情報を漏らさない一貫した画面へ変換します。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	root layout	root layout — 入力と前提を確認し、境界を曖昧にしない。
2	conditional nav	conditional nav — 値がどの順で変換されるかを追跡する。
3	form errors	form errors — 失敗を再現し、型またはログから原因を特定する。
4	safe error page	safe error page — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

layout・auth pages・error viewsとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 07 · MINIMUM CODE

まず動く最小コード

layout・auth pages・error viewsだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[layout("/")]async fn root(cx:&Cx,slot:Result)->Result{let user=current_user(cx).await?;view!{<!DOCTYPE html><html lang="ja"><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-scale=1"><link rel="stylesheet" href=(CSS)></head><body><nav>if user.is_some(){<a href="/notes">"ノート"</a>}else{<a href="/login">"ログイン"</a>}</nav><main id="main">(slot?)</main></body></html>}}
```

● ● ● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 07 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[layout("/")]</code> <code>async fn root(cx:&Cx,slot:Result)->Result{let user=current_u</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>layout</code> ・ <code>auth</code> <code>pages</code> ・ <code>error</code> <code>views</code>	入力、処理、出力を分離して読みます。
<code>layout</code> ・ <code>auth</code> <code>pages</code> ・ <code>error</code> <code>views</code>	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力が`root` `layout`の条件を満たすか確認する。
- ✓ `conditional nav`を実行し、途中の型と状態を追う。
- ✓ `form errors`で失敗を成功経路から分離する。
- ✓ `safe error page`により、出力と副作用を検証する。

型を声に出す `layout`・`auth` `pages`・`error` `views`に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 07・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

`layout`・`auth` `pages`・`error` `views`

未認証・認証済みで`nav`が切り替わり、`form error`と`focus`が安全かつ読みやすく表示されます。

観察ポイント

- `root layout`
- `conditional nav`
- `form errors`

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 07・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
<form method="post" action="/login"><div role="alert" tabindex="-1">(summary)</div><label for="email">メール</label><input id="email" name="email" type="email" autocomplete="username"><label for="password">パスワード</label><input id="password" name="password" type="password" autocomplete="current-password"><button>ログイン</button></form>
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

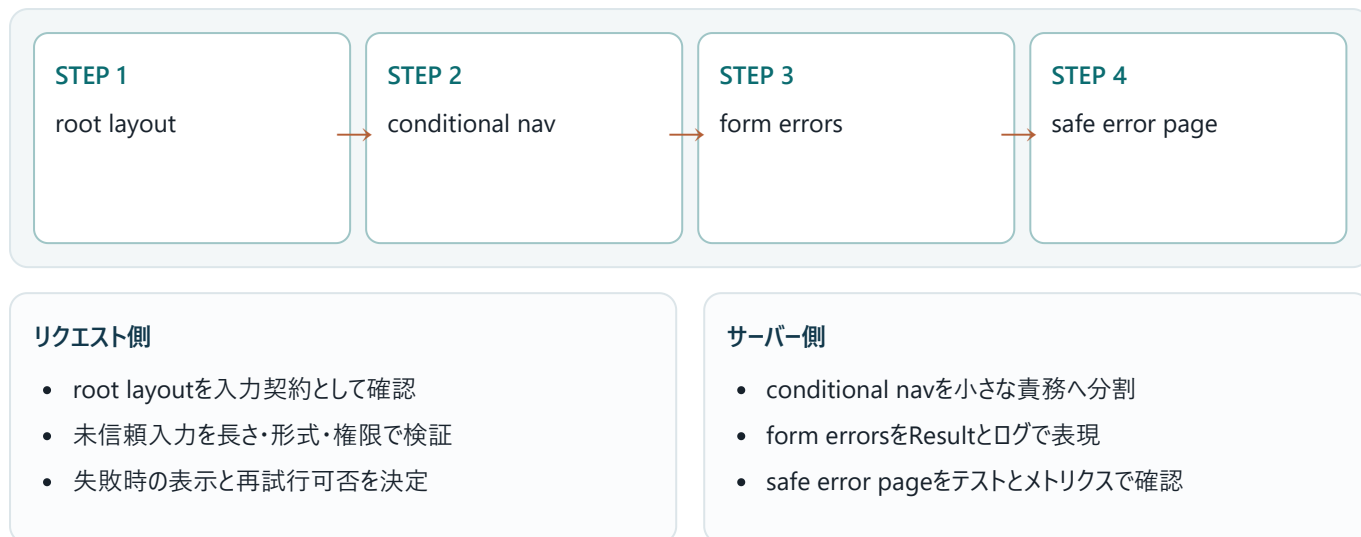
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 07・BACKEND APPLICATION

バックエンドのどこで使うか

component引数はescapeされ、raw HTMLを作りません。error detailはlogへ、画面は安全な定型文と再試行導線を出します。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 07 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	root layoutに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	conditional navの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	form errorsまたはsafe error pageの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 07・DEBUGGING

layout・auth pages・error viewsを再現して切り分ける

layout・auth pages・error viewsの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: root layout
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: conditional nav
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: form errors
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: safe error page

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	root layout	root layout — 入力と前提を確認し、境界を曖昧にしない。
2	conditional nav	conditional nav — 値がどの順で変換されるかを追跡する。
3	form errors	form errors — 失敗を再現し、型またはログから原因を特定する。
4	safe error page	safe error page — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 layoutでdocumentとnavを共通化し、form/errorは安全な値・focus・semantic HTMLで描きます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 07 · PRACTICE

手を動かす演習

login formをlabel、autocomplete、error summary、aria-describedby付きで書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 07 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
<form method="post" action="/login"><div role="alert" tabindex="-1">(summary)</div><label for="email">メール</label><input id="email" name="email" type="email" autocomplete="username"><label for="password">パスワード</label><input id="password" name="password" type="password" autocomplete="current-password"><button>ログイン</button></form>
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ layoutでdocumentとnavを共通化し、form/errorは安全な値・focus・semantic HTMLで描きます。
- ✓ layout・auth pages・error viewsとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 08

CRUD routesと画面の統合

UNIT 08 · CONCEPT

CRUD routesと画面の統合

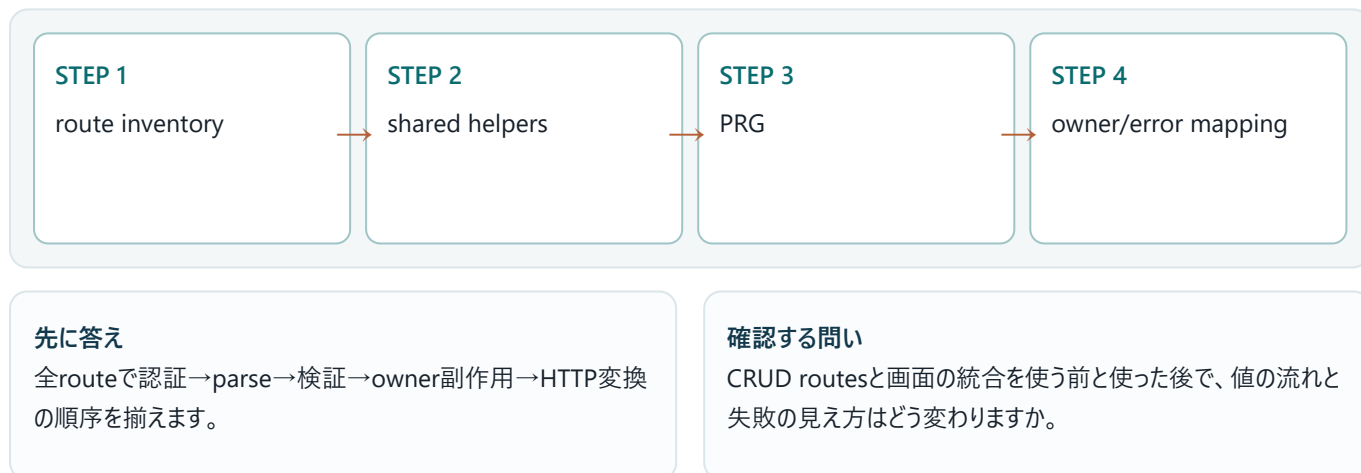
index/create/show/edit/update/toggle/deleteをRoute inventoryどおり実装します。各routeでrequire_user、typed ID、validation、owner Repository、303/400/404/409を同じ順で適用します。

この単元の到達点

- route inventory
- shared helpers
- PRG
- owner/error mapping

なぜバックエンドで必要か

current_user、note_id、repo、form componentをhelper化しますが、routeのHTTP意味が読めなくなる過度なmacro化は避けます。



UNIT 08 · MENTAL MODEL

CRUD routesと画面の統合を図でつかむ



index/create/show/edit/update/toggle/deleteをRoute inventoryどおり実装します。各routeでrequire_user、typed ID、validation、owner Repository、303/400/404/409を同じ順で適用します。図では左から右へ処理を追います。各箱の入口では入力の種類と信頼度、出口では結果の種類と副作用を確認します。

順序	観察対象	初心者が確認すること
1	route inventory	route inventory — 入力と前提を確認し、境界を曖昧にしない。
2	shared helpers	shared helpers — 値がどの順で変換されるかを追跡する。
3	PRG	PRG — 失敗を再現し、型またはログから原因を特定する。
4	owner/error mapping	owner/error mapping — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

CRUD routesと画面の統合とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 08 · MINIMUM CODE

まず動く最小コード

CRUD routesと画面の統合だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[route(POST "/notes/{note_id}/toggle")]async fn toggle(cx:&Cx)->Result<SeeOther>{let
user=require_user(cx).await?;let
id=note_id(cx)?;repo(cx).toggle_owned(user.id,id).await?;Ok(see_other("/notes"))}
#[route(POST "/notes/{note_id}/delete")]async fn delete(cx:&Cx)->Result<SeeOther>{let
user=require_user(cx).await?;let
id=note_id(cx)?;repo(cx).delete_owned(user.id,id).await?;Ok(see_other("/notes"))}
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 08 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[route(POST "/notes/{note_id}/toggle")]async fn toggle(cx:&Cx)->Result<Se</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>#[route(POST "/notes/{note_id}/delete")]async fn delete(cx:&Cx)->Result<Se</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
CRUD routesと画面の統合	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力がroute inventoryの条件を満たすか確認する。
- ✓ shared helpersを実行し、途中の型と状態を追う。
- ✓ PRGで失敗を成功経路から分離する。
- ✓ owner/error mappingにより、出力と副作用を検証する。

型を声に出す CRUD routesと画面の統合に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 08 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

CRUD routesと画面の統合

登録からCRUD、logoutまで同じdesign systemで動き、Network statusとDB変化が教材の表と一致します。

観察ポイント

- route inventory
- shared helpers
- PRG

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 08・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
POST create: Form→INSERT→303; 400/503
POST update: Form+Id+Version→UPDATE→303; 400/404/409/503
POST toggle: Id→atomic UPDATE→303; 400/404/503
POST delete: Id→DELETE→303; 400/404/503
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 08・BACKEND APPLICATION

バックエンドのどこで使うか

current_user、note_id、repo、form componentをhelper化しますが、routeのHTTP意味が読めなくなる過度なmacro化は避けます。



リクエスト側

- route inventoryを入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- shared helpersを小さな責務へ分割
- PRGをResultとログで表現
- owner/error mappingをテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 08・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	route inventoryに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	shared helpersの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	PRGまたはowner/error mappingの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 08・DEBUGGING

CRUD routesと画面の統合を再現して切り分ける

CRUD routesと画面の統合の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: route inventory
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: shared helpers
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: PRG
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: owner/error mapping

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	route inventory	route inventory — 入力と前提を確認し、境界を曖昧にしない。
2	shared helpers	shared helpers — 値がどの順で変換されるかを追跡する。
3	PRG	PRG — 失敗を再現し、型またはログから原因を特定する。
4	owner/error mapping	owner/error mapping — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 全routeで認証→parse→検証→owner副作用→HTTP変換の順序を揃えます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 08 ・ PRACTICE

手を動かす演習

各routeを入力・副作用・成功・失敗で表にし、test名を一つずつ対応させてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 08 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
POST create: Form→INSERT→303; 400/503
POST update: Form+Id+Version→UPDATE→303; 400/404/409/503
POST toggle: Id→atomic UPDATE→303; 400/404/503
POST delete: Id→DELETE→303; 400/404/503
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 全routeで認証→parse→検証→owner副作用→HTTP変換の順序を揃えます。
- ✓ CRUD routesと画面の統合とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 09

CSS・asset・responsive・a11y

UNIT 09 ・ CONCEPT

CSS・asset・responsive・a11y

app.cssはtoken、layout、form、button、card、status、focus、mobileをまとめ、asset!でcontent hash bundleします。HTML構造と組み合わせてkeyboardとscreen readerを支えます。

この単元の到達点

- CSS tokens
- component states
- focus-visible
- responsive

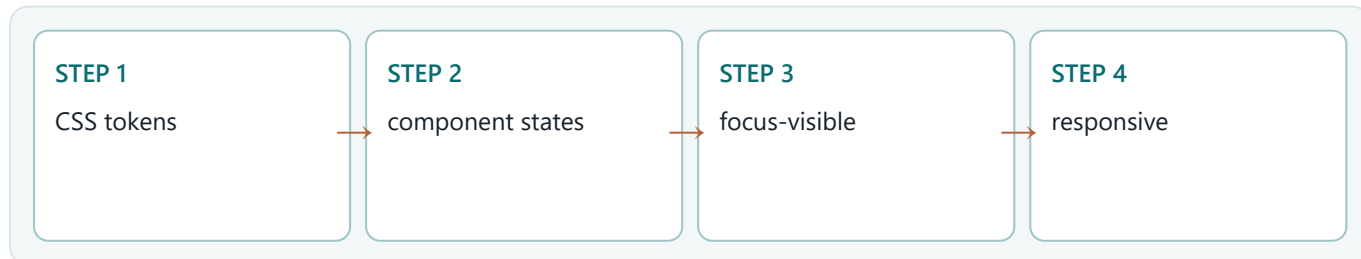
なぜバックエンドで必要か

色だけでdone/errorを伝えずtextとsemantic属性を併用します。prefers-reduced-motionとcontrastも確認します。



UNIT 09 ・ MENTAL MODEL

CSS・asset・responsive・a11yを図でつかむ



app.cssはtoken、layout、form、button、card、status、focus、mobileをまとめ、asset!でcontent hash bundleします。HTML構造と組み合わせてkeyboardとscreen readerを支えます。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	CSS tokens	CSS tokens — 入力と前提を確認し、境界を曖昧にしない。
2	component states	component states — 値がどの順で変換されるかを追跡する。
3	focus-visible	focus-visible — 失敗を再現し、型またはログから原因を特定する。
4	responsive	responsive — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

CSS・asset・responsive・a11yとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 09・MINIMUM CODE
まず動く最小コード

CSS・asset・responsive・a11yだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
:root{--bg:#f4f7f7;--surface:#fff;--ink:#102a3b;--accent:#0b7c7d;--danger:#a83232;--radius:.75rem}body{margin:0;background:var(--bg);color:var(--ink);font-family:system-ui,sans-serif}.container{width:min(68rem,calc(100% - 2rem));margin:auto}.note-card{background:var(--surface);border:1px solid #d9e4e6;border-radius:var(--radius);padding:1rem}:focus-visible{outline:3px solid #ef9d62;outline-offset:3px}@media(max-width:42rem){.actions{display:grid;grid-template-columns:1fr 1fr}}
```

●●● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 09・LINE BY LINE
コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>:root{--bg:#f4f7f7;--surface:#fff;--ink:#102a3b;--accent:#0b7c7d;--danger:</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>CSS・asset・responsive・a11y</code>	入力、処理、出力を分離して読みます。
<code>CSS・asset・responsive・a11y</code>	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力がCSS tokensの条件を満たすか確認する。
- ✓ component statesを実行し、途中の型と状態を追う。
- ✓ focus-visibleで失敗を成功経路から分離する。
- ✓ responsiveにより、出力と副作用を検証する。

型を声に出す CSS・asset・responsive・a11yに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 09 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

CSS・asset・responsive・a11y

nav、form、note card、statusがdesktop/mobileで整い、keyboard focusが常に見えます。

観察ポイント

- CSS tokens
- component states
- focus-visible

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 09・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
320: no x-scroll/44px targets/form one column
768: readable line length/actions wrap
1280: max width/not stretched
All: visible focus/logical heading/label/name/status/error
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

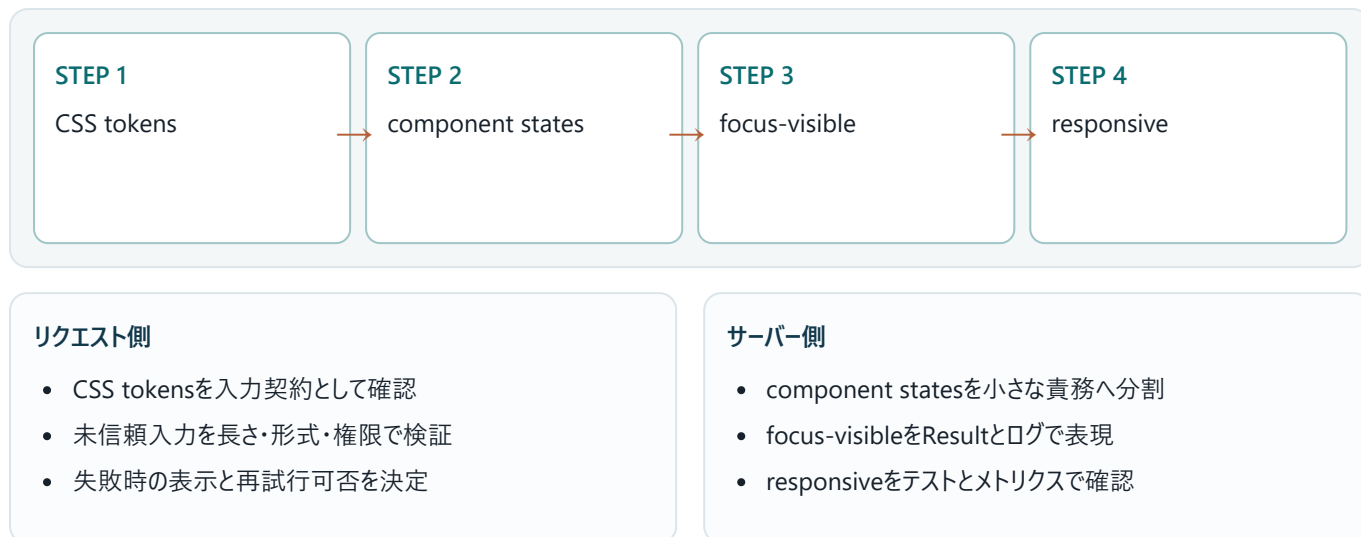
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 09・BACKEND APPLICATION

バックエンドのどこで使うか

色だけでdone/errorを伝えずtextとsemantic属性を併用します。prefers-reduced-motionとcontrastも確認します。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 09・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	CSS tokensに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	component statesの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	focus-visibleまたはresponsiveの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 09・DEBUGGING

CSS・asset・responsive・a11yを再現して切り分ける

CSS・asset・responsive・a11yの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: CSS tokens
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: component states
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: focus-visible
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: responsive

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	CSS tokens	CSS tokens — 入力と前提を確認し、境界を曖昧にしない。
2	component states	component states — 値がどの順で変換されるかを追跡する。
3	focus-visible	focus-visible — 失敗を再現し、型またはログから原因を特定する。
4	responsive	responsive — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 CSS assetとsemantic HTMLと一緒に検証し、見た目と操作可能性を同時に完成させます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 09 ・ PRACTICE

手を動かす演習

Playwright等のscreen size 320/768/1280でoverflowとTab操作を確認する項目を書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 09・SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
320: no x-scroll/44px targets/form one column
768: readable line length/actions wrap
1280: max width/not stretched
All: visible focus/logical heading/label/name/status/error
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ CSS assetとsemantic HTMLと一緒に検証し、見た目と操作可能性を同時に完成させます。
- ✓ CSS・asset・responsive・a11yとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 10

tests・Docker・fly.toml・CI・runbook

UNIT 10・CONCEPT

tests・Docker・fly.toml・CI・runbook

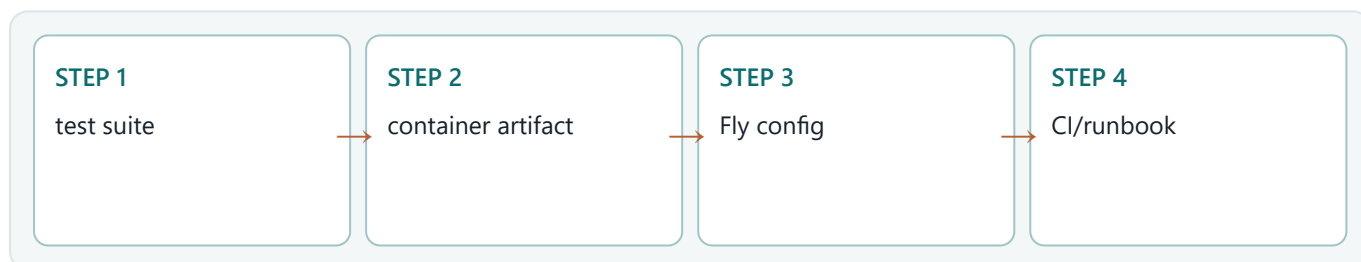
完全コードの最後に、unit/integration/security/smoke、multi-stage Docker、target/assets、Fly port/check、CI gate、deploy/rollback手順を同じ版へ揃えます。

この単元の到達点

- test suite
- container artifact
- Fly config
- CI/runbook

なぜバックエンドで必要か

同じcommandをlocalとCIで使い、deploy後はversion/digest/check/log/auth/CRUDをsmokeします。rollbackはprevious imageとschema互換を先に確認します。



先に答え

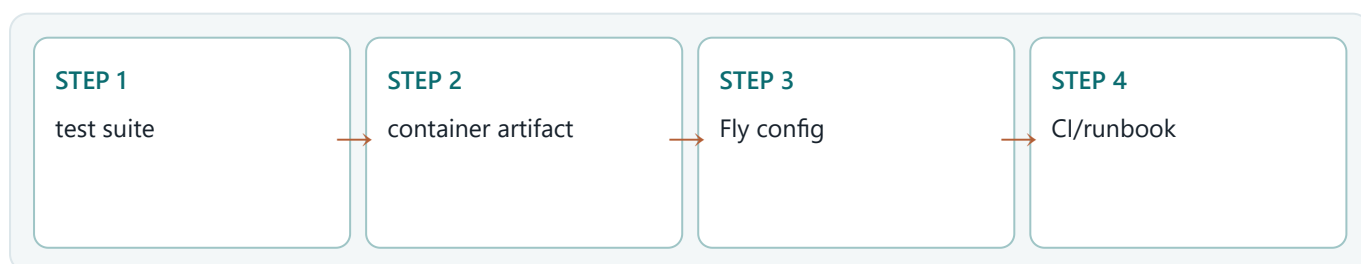
完全コードはsourceだけでなく、test・asset・container・config・運用手順まで含めて完成です。

確認する問い

tests・Docker・fly.toml・CI・runbookを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

UNIT 10・MENTAL MODEL

tests・Docker・fly.toml・CI・runbookを図でつかむ



完全コードの最後に、unit/integration/security/smoke、multi-stage Docker、target/assets、Fly port/check、CI gate、deploy/rollback手順を同じ版へ揃えます。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と

副作用を確認します。

順序	観察対象	初心者が確認すること
1	test suite	test suite — 入力と前提を確認し、境界を曖昧にしない。
2	container artifact	container artifact — 値がどの順で変換されるかを追跡する。
3	Fly config	Fly config — 失敗を再現し、型またはログから原因を特定する。
4	CI/runbook	CI/runbook — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

tests・Docker・fly.toml・CI・runbookとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 10・MINIMUM CODE

まず動く最小コード

tests・Docker・fly.toml・CI・runbookだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
cargo fmt --all -- --check
cargo clippy --all-targets --all-features -- -D warnings
cargo test --all-features
cargo build --locked --release
topcoat asset bundle --release
docker build -t rustnote:$GIT_SHA .
docker run --rm -p 8080:8080 -e HOST=0.0.0.0 -e PORT=8080 rustnote:$GIT_SHA
curl -f http://127.0.0.1:8080/health
fly deploy
```

●●● 実行コマンド

```
cargo test --all-features
docker build -t rustnote:verify .
fly deploy
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 10・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>cargo fmt --all -- --check</code>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
<code>cargo clippy --all-targets --all-features -- -D warnings</code>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
<code>cargo test --all-features</code>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
<code>cargo build --locked --release</code>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
<code>topcoat asset bundle --release</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力が入力test suiteの条件を満たすか確認する。
- ✓ container artifactを実行し、途中の型と状態を追う。
- ✓ Fly configで失敗を成功経路から分離する。
- ✓ CI/runbookにより、出力と副作用を検証する。

型を声に出す tests・Docker・fly.toml・CI・runbookに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 10 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

tests pass
container health 200
Fly checks passing
public auth/CRUD smoke passing

見える結果
端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと
型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 10・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
Pre: green gates/backup/schema compatibility/secrets
During: release migration/checks/logs
Post: version/health/assets/auth/CRUD/SLO
Rollback: previous digest/schema compatible/smoke/incident timeline
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 10・BACKEND APPLICATION

バックエンドのどこで使うか

同じcommandをlocalとCIで使い、deploy後はversion/digest/check/log/auth/CRUDをsmokeします。rollbackはprevious imageとschema互換を先に確認します。



リクエスト側

- test suiteを入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- container artifactを小さな責務へ分割
- Fly configをResultとログで表現
- CI/runbookをテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 10 ・ FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	test suiteに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	container artifactの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	Fly configまたはCI/runbookの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 10・DEBUGGING

tests・Docker・fly.toml・CI・runbookを再現して切り分ける

tests・Docker・fly.toml・CI・runbookの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: test suite
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: container artifact
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: Fly config
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: CI/runbook

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	test suite	test suite — 入力と前提を確認し、境界を曖昧にしない。
2	container artifact	container artifact — 値がどの順で変換されるかを追跡する。
3	Fly config	Fly config — 失敗を再現し、型またはログから原因を特定する。
4	CI/runbook	CI/runbook — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 完全コードはsourceだけでなく、test・asset・container・config・運用手順まで含めて完成です。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 10 ・ PRACTICE

手を動かす演習

release checklistを事前・deploy中・事後・rollbackの四段階で完成させてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 10 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

Pre: green gates/backup/schema compatibility/secrets

During: release migration/checks/logs

Post: version/health/assets/auth/CRUD/SLO

Rollback: previous digest/schema compatible/smoke/incident timeline

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 完全コードはsourceだけでなく、test・asset・container・config・運用手順まで含めて完成です。
- ✓ tests・Docker・fly.toml・CI・runbookとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

CAPSTONE

完全なRustNoteをbuild・test・container smoke・Fly公開する

10単元を一つの成果物へ統合します。動く結果だけでなく、途中のエラー、設計判断、確認コマンドも提出物です。

順	使う単元	統合する判断
01	Cargo.tomlと実行可能な版固定	完全コードはmanifest・lock・CLI版・実行コマンドを不可分にします。
02	migrations全体とschema不変条件	アプリ検証をすり抜けてもDB制約が不正状態を拒否します。
03	Config・main・router bootstrap	mainは依存を組み立てるcomposition rootに限定します。
04	domain型とAppError	検証済み値と失敗分類をdomain型にし、framework表現は外側で変換します。
05	libSQL NoteRepository完全経路	Repositoryの全SQLへowner bindを含め、外へdomain型と分類済みerrorだけを返します。
06	password・session・current_user	資格情報検証、cookie token、DB hash、期限、revokeを一つの認証経路にします。
07	layout・auth pages・error views	layoutでdocumentとnavを共通化し、form/errorは安全な値・focus・semantic HTMLで描きます。
08	CRUD routesと画面の統合	全routeで認証→parse→検証→owner副作用→HTTP変換の順序を揃えます。
09	CSS・asset・responsive・a11y	CSS assetとsemantic HTMLと一緒に検証し、見た目と操作可能性を同時に完成させます。
10	tests・Docker・fly.toml・CI・runbook	完全コードはsourceだけでなく、test・asset・container・config・運用手順まで含めて完成です。

完了条件

- ✓ manifest/lock/migration/source/assets/configが揃う
- ✓ 全コード片の出力と画面を確認した
- ✓ securityとowner testが通る
- ✓ 公開後smokeとrollback手順を実施した

REVIEW

巻末確認問題

用語だけでなく、「小さな例」「バックエンドでの場所」「失敗時の挙動」を含めて教えてください。

1. **01.** Cargo.tomlと実行可能な版固定を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
2. **02.** migrations全体とschema不変条件を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
3. **03.** Config・main・router bootstrapを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
4. **04.** domain型とAppErrorを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
5. **05.** libSQL NoteRepository完全経路を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
6. **06.** password・session・current_userを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
7. **07.** layout・auth pages・error viewsを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
8. **08.** CRUD routesと画面の統合を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
9. **09.** CSS・asset・responsive・a11yを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
10. **10.** tests・Docker・fly.toml・CI・runbookを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

答える順序 まず資料を閉じて説明し、次に最小コードを書き、最後に該当単元へ戻って不足を補います。正解の暗記ではなく、根拠を再現できることが目標です。

REVIEW ANSWERS

確認問題・解答の骨格

次の要点は模範解答の丸暗記用ではありません。自分の説明に「定義・小さな例・バックエンドでの場所」が含まれているかを照合します。

01. Cargo.tomlと実行可能な版固定

Cargo.tomlと実行可能な版固定とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 完全コードはmanifest・lock・CLI版・実行コマンドを不可分にします。

02. migrations全体とschema不変条件

migrations全体とschema不変条件とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: アプリ検証をすり抜けてもDB制約が不正状態を拒否します。

03. Config・main・router bootstrap

Config・main・router bootstrapとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: mainは依存を組み立てるcomposition rootに限定します。

04. domain型とAppError

domain型とAppErrorとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 検証済み値と失敗分類をdomain型にし、framework表現は外側で変換します。

05. libSQL NoteRepository完全経路

libSQL NoteRepository完全経路とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: Repositoryの全SQLへowner bindを含め、外へdomain型と分類済みerrorだけを返します。

06. password・session・current_user

password・session・current_userとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 資格情報検証、cookie token、DB hash、期限、revokeを一つの認証経路にします。

07. layout・auth pages・error views

layout・auth pages・error viewsとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: layoutでdocumentとnavを共通化し、form/errorは安全な値・focus・semantic HTMLで描きます。

08. CRUD routesと画面の統合

CRUD routesと画面の統合とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 全routeで認証→parse→検証→owner副作用→HTTP変換の順序を揃えます。

09. CSS・asset・responsive・a11y

CSS・asset・responsive・a11yとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: CSS assetとsemantic HTMLと一緒に検証し、見た目と操作可能性を同時に完成させます。

10. tests・Docker・fly.toml・CI・runbook

tests・Docker・fly.toml・CI・runbookとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 完全コードはsourceだけでなく、test・asset・container・config・運用手順まで含めて完成です。

GLOSSARY

重要用語と実務での位置

用語	一文定義	バックエンドでの位置
Cargo.tomlと実行可能な版固定	完全コードはmanifest・lock・CLI版・実行コマンドを不可分にします。	教材のコードはこのmanifestとlockで検証します。版を外した場合はdocsとAPI差を確認し、同じ結果を保証しません。
migrations全体とschema不変条件	アプリ検証をすり抜けてもDB制約が不正状態を拒否します。	起動時runnerは適用済みversionを読み、未適用SQLだけをtransactionで実行します。本番はrelease commandから一度だけ実行します。
Config・main・router bootstrap	mainは依存を組み立てるcomposition rootに限定します。	Topcoat startはHOST/PORTを読みます。Flyではenvに0.0.0.0/8080を設定し、AppStateにはSend+SyncなDB handleと検証済みConfigだけを置きます。
domain型とAppError	検証済み値と失敗分類をdomain型にし、framework表現は外側で変換します。	errorのDisplayは内部診断、利用者向けmessageはweb mappingで定型化します。source連鎖とrequest_idはlogへ残します。
libSQL NoteRepository完全経路	Repositoryの全SQLへowner bindを含め、外へdomain型と分類済みerrorだけを返します。	Connection取得、transaction、query timeout、operation spanをRepository内部で統一します。libsql error文字列をHTMLへ出しません。
password・session・current_user	資格情報検証、cookie token、DB hash、期限、revokeを一つの認証経路にします。	password hash、session raw token、Turso tokenをlogへ出しません。login rate limitとsecurity testsをrelease gateにします。
layout・auth pages・error views	layoutでdocumentとnavを共通化し、form/errorは安全な値・focus・semantic HTMLで描きます。	component引数はescapeされ、raw HTMLを作りません。error detailはlogへ、画面は安全な定型文と再試行導線を出します。
CRUD routesと画面の統合	全routeで認証→parse→検証→owner副作用→HTTP変換の順序を揃えます。	current_user、note_id、repo、form componentをhelper化しますが、routeのHTTP意味が読めなくなる過度なmacro化は避けます。
CSS・asset・responsive・a11y	CSS assetとsemantic HTMLと一緒に検証し、見た目と操作可能性を同時に完成させます。	色だけでdone/errorを伝えずtextとsemantic属性を併用します。prefers-reduced-motionとcontrastも確認します。
tests・Docker・fly.toml・CI・runbook	完全コードはsourceだけでなく、test・asset・container・config・運用手順まで含めて完成です。	同じcommandをlocalとCIで使い、deploy後はversion/digest/check/log/auth/CRUDをsmokeします。rollbackはprevious imageとschema互換を先に確認します。

PRIMARY SOURCES

公式資料と更新確認

仕様は更新されます。教材で意図を理解した後、実装時点の一次資料でAPI、security note、breaking changeを確認してください。

1. Rust Book

<https://doc.rust-lang.org/stable/book/>
Rust 2024 Edition を前提にした公式入門書

2. Rust Reference

<https://doc.rust-lang.org/reference/>
言語仕様を確認する一次資料

3. Standard Library

<https://doc.rust-lang.org/std/>
標準ライブラリAPI

4. Cargo Book

<https://doc.rust-lang.org/cargo/>
ビルド、依存関係、workspace、features

5. Rustdoc Book

<https://doc.rust-lang.org/rustdoc/>
APIドキュメントの書き方

6. Clippy

<https://doc.rust-lang.org/clippy/>
Rust公式Lint

7. Async Book

<https://rust-lang.github.io/async-book/>
非同期Rustの公式解説

8. Tokio Tutorial

<https://tokio.rs/tokio/tutorial>
Tokio公式チュートリアル

9. Topcoat

<https://github.com/tokio-rs/topcoat>
Topcoat公式リポジトリ。early-stageの注意を含む

10. Topcoat 0.5.0 API

<https://docs.rs/topcoat/0.5.0/topcoat/>
本教材で固定するAPI

11. Turso Rust SDK

<https://docs.turso.tech/sdk/rust/quickstart>
libSQL Rust SDK公式手順

12. Better Auth Database

<https://better-auth.com/docs/concepts/database>
Better AuthのDB構成

13. Fly.io Deploy

<https://fly.io/docs/launch/deploy/>
fly launch / fly deploy公式手順

14. Fly.io Health Checks

<https://fly.io/docs/reference/health-checks/>
公開後のヘルスチェック

15. OWASP Cheat Sheets

<https://cheatsheetseries.owasp.org/>
Webセキュリティの実務チェック

採用判断 本教材はRust 2024 Edition、Topcoat 0.5.0固定、Turso/libSQL、Topcoat Session + Argon2id、Fly.ioを主経路にします。Better AuthはTypeScript sidecarが適切な要件で比較します。